

NAME: DANIYAL SAEED

SAP ID: 53937

SECTION: BS Data Science

SUBJECT: COAL

ASSIGNMENT NO 4

SOL: (A) Detailed Programmed I/O Routine for Reading the High-Precision Pressure Sensor

Assumptions for clarity

- Device A (pressure sensor) is connected through its own I/O module.
- The I/O module provides two important registers:
 - STATUS Register (Port = PORT_STATUS_A)
 - Bit0 = Data Ready (1 when new pressure reading is available)
 - DATA Register (Port = PORT_DATA_A)
 - Contains the 16-bit pressure reading
- CPU uses I/O mapped I/O, so it accesses these ports using IN instruction.
- CPU must keep reading the status register until new data is available → polling loop.

Long & Detailed Pseudocode (Driver Routine Using Programmed I/O)

Routine: ReadPressureSensor_ProgrammedIO

1. BEGIN
2. // Step 1: Start by checking the sensor's status register
3. status $\leftarrow$ IN(PORT_STATUS_A)
4. // Step 2: Poll the Data Ready flag (bit 0)
5. WHILE ((status AND 0001h) = 0) DO
6. // CPU keeps reading the status register repeatedly
7. status $\leftarrow$ IN(PORT_STATUS_A)
8. END WHILE
9. // Step 3: When Data Ready = 1, CPU reads the 16-bit data register
10. pressureValue $\leftarrow$ IN(PORT_DATA_A)
11. // Step 4: CPU may store the pressure value into memory
12. MEMORY[PRESSURE_BUFFER] $\leftarrow$ pressureValue

13. // Step 5: Return the value to the calling system

14. RETURN pressureValue

15. END

Detailed Step-By-Step Flowchart Explanation

Step 1 — CPU Initiates Read Operation

- The CPU executes an IN instruction to read the STATUS register of the pressure sensor.
- This register tells the CPU whether a new pressure reading is ready.

Step 2 — Check the Data-Ready Flag

- The CPU examines bit 0 of the STATUS register.
- If this bit is 0, it means:
 - The sensor has *not* produced a new reading yet.
 - The CPU must wait.

Step 3 — Polling Loop

- Because Programmed I/O uses polling, the CPU must:
 - Continuously read the STATUS register again and again.
 - Check the Data Ready flag each time.
- This loop continues until the sensor finally updates the data and sets the ready bit to 1.

Step 4 — Data Is Ready

- Once the Data Ready bit becomes 1:
 - The CPU knows the sensor has produced a fresh 16-bit value.
 - The loop ends.

Step 5 — CPU Reads the Sensor Data

- CPU issues another IN instruction, now to the DATA register.
- The 16-bit pressure reading is transferred from the sensor's I/O module into the CPU via the data bus.

Step 6 — Store or Process Data

- CPU can store the data in memory (e.g., sensor buffer).
- Or pass it to another routine for processing or display.

Step 7 — End of Routine

- Routine returns the pressure reading to the calling program.
- The CPU continues its next task.

(B) Main Disadvantage of Programmed I/O for This Slow Sensor

Below is an expanded, high-quality explanation.

Disadvantage: CPU Wastes Processing Time (Busy Waiting / Polling Overhead)

In Programmed I/O, the CPU performs polling, meaning:

- It repeatedly reads the device's status register in a loop.
- It cannot move on to other tasks until the device becomes ready.
- It does nothing productive during this time.

If the pressure sensor is slow to update:

- The CPU may run the polling loop hundreds or thousands of times.
- Each iteration consumes CPU cycles.
- The CPU becomes completely occupied waiting for the sensor.

Why This Reduces System Performance

1. CPU Is Locked in a Loop

- Instead of performing useful work (motor control, display updates, data logging), it is stuck checking a single status bit.

2. Wasted Clock Cycles

- Every IN instruction and comparison uses CPU time.
- All these cycles are wasted because the sensor will only update after a fixed slow interval.

3. System Becomes Less Responsive

- If there are multiple I/O devices, they may experience delays.
- Real-time control tasks may be interrupted or slowed down.

4. Inefficient Use of Resources

- Programmed I/O does not use interrupts or DMA.
- There is no mechanism for the sensor to “notify” the CPU.
- CPU efficiency drops significantly in slow-device scenarios.

Final Summary

The primary disadvantage of using Programmed I/O is that the CPU must continuously poll the sensor's status register, checking the Data Ready flag in a loop.

If the pressure sensor has a slow update rate, the CPU wastes large amounts of time performing useless status checks. This leads to busy-waiting, unnecessary consumption of CPU cycles, and reduced overall system efficiency. The CPU cannot perform other important tasks during this wait, which decreases the performance of the entire industrial monitoring system.